

Procesadores del Lenguaje

Curso 2024-2025

Práctica Final – Segunda Parte

Traductor de Expresiones a un Lenguaje en Notación Prefija

Esta segunda parte de la práctica continúa ampliando las especificaciones para permitir la traducción de un pequeño subconjunto de código C. Esto incluirá estructuras condicionales (if, while, for), funciones y matrices. La salida seguirá estando en notación prefija, que corresponde a una variante del lenguaje Lisp, que utilizaremos como lenguaje intermedio.

En futuras partes también se abordará el diseño de un pequeño backend para traducir de Lisp a un lenguaje de bajo nivel para una máquina de pila (como alternativa a los cpus más convencionales programados con Risc V o lenguaje ensamblador x86).

Trabajo a realizar:

1. Leed el enunciado pasado completo antes de comenzar.
2. Revisad las especificaciones de éste documento con anotaciones añadidas resaltadas. Comprobad que vuestro trabajo ya realizado sigue dichas especificaciones.
3. **Renombrad vuestro fichero de código en sucesivas sesiones a trad2.y, trad3.y, ...**
4. Continúad con el trabajo en el punto que corresponda, desarrollando los puntos indicados en las especificaciones hasta donde os de tiempo. En futuras sesiones continuaremos con ellas.
5. Comprobad vuestra instalación local de CLisp.
6. Podéis evaluar los resultados de traducción usando el intérprete online https://rextester.com/l/common_lisp_online_compiler. Para ello podéis:
 - a. editar un fichero (por ejemplo, **prueba.c**) con una serie de expresiones
 - b. ejecutar **cat prueba.c | ./trad >prueba.l**
 - c. copiar la salida y pegarla en la ventana del intérprete
 - d. con F8 se compila y ejecuta.

Entrega de Sesión:

Seguid las instrucciones del entregador.

- Un fichero **trad2.y** con el trabajo realizado hasta el momento. Includid en la cabecera del fichero dos líneas de comentario iniciales. La primera con vuestros nombres y número de grupo. Y la segunda con los correos electrónicos (separados con un espacio).
- Un documento llamado **trad2.pdf** con una breve explicación del trabajo realizado.

Después de hacer la entrega, descargad vuestro fichero y comprobad que funciona correctamente y que cumple con las indicaciones.

CUESTIONES IMPORTANTES:

Los errores de entrega influirán en la evaluación de la práctica. Esto incluye las prácticas no entregadas.

No se valorarán aportaciones que no hayan sido acordadas previamente con los profesores.

De cara a la evaluación de ésta práctica tened en cuenta que la dificultad será incremental conforme avancen los puntos planteados. Los primeros se pueden resolver en clase por lo que también contarán menos. En las últimas sesiones se valorará vuestra autonomía a la hora de completar los puntos propuestos.

No se recomienda en ningún caso dejar la práctica para última hora. Las sesiones de prácticas están pensadas para dar soporte con los errores que surgen al usar *bison*.

#1

Si se desea generar una salida Lisp formateada, no se debe intentar desde la semántica del parser, puesto que eso la complica y dificulta su comprensión.

Una buena solución consiste en programar una función backend específica que recibe la cadena traducida para imprimir y que la vaya recorriendo y generando el formato deseado a la vez que la imprime.

#2

Revisad en el código **yylex()** la utilidad de la directiva de código embebido **//@**. Tendrá un papel fundamental en las pruebas de evaluación.

Para lanzarlas es necesario recurrir a una instrucción **//@ (main)** que estará incluida al final del programa de prueba.

Ejemplo:

```
int a = 10 ;
int b = 23 ;

f1 (int a, int b) {
    return a+b ;
}
//@ (print (f1 2 3))          ; permite probar f1 desde el código fuente

main () {
    printf ("%d", f1 (a, b)) ;
}
//@ (main)                   ; permite lanzar la ejecución del programa
```

#3

Debido a lo anterior se prohíbe generar de forma automática la salida **(main)** en la traducción ya que esto provoca dobles ejecuciones y otros problemas.

Tampoco debéis realizar llamadas a **exit()** cuando consideréis terminado el programa. Esto sucede de forma automática cuando **yylex** lee un *EOF* y devuelve **return(0)** a **yyparse**.

Especificaciones (Frontend)

Las especificaciones previas del fichero **trad1.y** proporcionado son:

- Hay una estructura jerárquica en la gramática que empieza con:
 - **Axioma**, se encarga de la recursividad (**r_axioma**) y deriva:
 - **Sentencia**, que a su vez deriva las asignaciones y las impresión de expresiones.
- La sentencia que contiene únicamente una expresión se ha eliminado. Aunque en el lenguaje C se permiten sentencias como **1+2**; no les sacaremos partido, y además pueden ser una fuente de conflictos.
- Podéis añadir vuestra solución para asignaciones encadenadas si disponéis de ella.
- El código de **trad1.y** utiliza código diferido. Dejaremos el posible uso de AST para más adelante.
- Estudiad las soluciones proporcionadas para:
 - a. **termino ::= operando**
 - b. **termino ::= + operand**
 - c. **termino ::= - operand**
 - d. **operando ::= IDENTIF**
 - e. **operando ::= (expresion)**

Proponemos una secuencia de pasos para desarrollar esta práctica. Se recomienda abordar cada uno de los pasos de forma secuencial.

1. Incluid la definición de variables globales en la gramática.
 - a. La definición en C será **int <id>;**² que debe traducirse a **(setq <id> 0)**. Es necesario incluir un segundo parámetro en Lisp para inicializar las variables. En el caso de la inicialización más simple en C, este valor será 0 por omisión. Puede haber 0, 1 o más líneas con definiciones de variables cada una comenzando por **int**.
 - b. Ampliad la gramática para contemplar la definición de una variable con inicialización incluida: **int <id> = <cte> ;** que debe ser traducido a **(setq <id> <cte>)**. Utilizamos aquí **<cte>** para representar un valor numérico constante. No consideraremos por ahora expresiones en las declaraciones.
 - c. Ampliad la gramática para contemplar la definición múltiple de variables con asignaciones opcionales: **int <id1> = 3, <id2>, ..., <idk> = 1 ;** que debe ser traducido a una secuencia de definiciones individuales **(setq <id1> 3) (setq <id2> 0) ... (setq <idk> 1)**. El orden de impresión de las variables debe corresponder al de la declaración. Prestad atención a que se asignan valores constantes (numéricos), no expresiones evaluables. Esto es así porque para las variables globales en C no es posible evaluar dichas expresiones en tiempo de compilación.

Esto corresponde a la definición de variables globales. Recordamos que en C no está permitido inicializar variables con expresiones (con variables y funciones) en la instrucción en que es declarada puesto que el proceso se realiza en tiempo de compilación. La evaluación de expresiones se hace en tiempo de ejecución. Algunos compiladores sí disponen de una fase previa capaz de evaluar expresiones simples del tipo **int ab32 = 3+7 ;** y de sustituirlas por **int ab32=10;** pero sigue siendo una tarea realizada en tiempo de compilación.

² A partir de este punto comenzamos a traducir de C a Lisp

2. En C **main** es el procedimiento/función principal, Debe tener una palabra reservada en la tabla correspondiente, y debe enlazar con el *Token Main*. Ampliad la gramática para reconocer la función **main**. Debe permitir la inclusión de sentencias. Prestad atención al siguiente punto. Incluimos un ejemplo de traducción:

C	Lisp
<pre>int a ; main () { @ (a + 1) ; }</pre>	<pre>(setq a 0) (defun main () (print (+ a 1)))</pre>
	<pre>(main) ; Para ejecutar el programa</pre>

A través de la gramática es posible obligar a que exista una (única) función **main**.

Considerad que la estructura de un programa en C debe ser:

<Decl Variables> <Def Funciones>. En teoría debería poder intercalarse ambos tipos de definiciones, pero eso puede producir conflictos en el *parser*. Por ello seguiremos una estructura fija. Las sentencias que define la gramática sólo deben aparecer dentro del cuerpo de una función. Revisad la estructura de vuestra gramática. Debe estar diseñada de forma muy cuidadosa y estructurada, intentando que sea lo más jerárquica posible.

Aquí se indican varias cuestiones.

- 1) Hay que diseñar una gramática lo más jerárquica o estructurada posible. Los nombres de No Terminales deben escogerse con cuidado y deben ser representativos y significativos.
- 2) Esto evitará la aparición de errores típicos de diseños “enmarañados” o excesivamente complicados.
- 3) Se sugiere emplear una estructura de programa que empiece con la declaración de variables y luego con la definición de funciones.
- 4) La recomendación es definir las funciones en orden inverso de jerarquía, empezando con las funciones más sencillas y terminando por la principal, el **main**. Esto permitirá que más adelante el traductor tenga siempre referencia de las funciones que se usan porque ya se han definido previamente. Si se definen primero las funciones principales y luego las de inferior jerarquía, un compilador necesitará hacer averiguaciones sobre el tipo de las funciones que se llaman y que aún no se han definido, o emplear dos pasadas para realizar traducciones parciales y condicionadas.

El compilador de C requiere en estos casos de una declaración previa (prototipo) de las funciones. En Lisp deben definirse las funciones antes de ser usadas. Para evitarnos complicaciones con los prototipos usaremos programas en C con las funciones en orden jerárquico inverso.

3. Para imprimir cadenas literales proponemos: **puts(<string>);** que se convierte en **(print <string>)**. Ejemplo: **puts("Hola mundo");** que se convierte en **(print "Hola mundo")**. Comprobad que **yylex** detecta las secuencias de entrada entre dobles comillas y las envía como un *token String*.
4. Sustituid la sentencia para imprimir usando **@** por una nueva que comience con la palabra reservada **printf**. El formato de la impresión es **printf(<string>, <elem1>, ... , <elemN>);**
 - a. Comenzad por la versión simplificada **printf(<string>, <elem1>);** traduciéndola a **(princ <elem1>)**. Tomad nota que el contenido de la primera cadena, la cadena de formato, es complejo y requiere de un procesamiento muy elaborado para interpretarla que por ahora excede nuestros objetivos. Por ello debe ser reconocida por la gramática, pero no se traducirá de ninguna forma. Se omitirá en la traducción. El segundo parámetro **<elem1>** puede ser o bien una **<expresion>** o bien un **<string>**, y se traducirá con **(princ <elem1>)**
 - b. Ampliad la gramática para reconocer: **printf(<string>, <elem1>, ... , <elemN>);** El segundo y los siguientes parámetros **<elemX>** pueden ser o bien una **<expresion>** o bien un **<string>**, y se traducirán con **(princ <elemX>)** preservando el orden original.

La función **print** genera un salto de línea al final. La función **princ** omite dicho salto de línea y las dobles comillas de los strings..

5. Los operadores aritméticos, lógicos y de comparación en C se pueden combinar sin que el programador pueda apreciar matices particulares. En realidad se trata de operadores de diferente naturaleza. Los primeros devuelven valores numéricos, mientras que los lógicos y de comparación devuelven valores booleanos. Tendría sentido tratarlos por separado en la gramática, pero esto debe hacerse con cuidado dado que puede producir algunos conflictos **o perderse las precedencias definidas por las directivas de bison**. Los operadores lógicos y de comparación en Lisp se relacionan con los equivalentes en C en la siguiente tabla. Prestad atención a la asociatividad y precedencia definida para cada operador nuevo que incluyáis.

https://en.cppreference.com/w/c/language/operator_precedence

C	&&	 	!	!=	==	<	<=	>	>=	%
Lisp	and	or	not	/=	=	<	<=	>	>=	mod

6. Estructura de control **while (<expr>) { <codigo> } .** La traduciremos con la estructura **(loop while <expr> do <codigo>)** . Prestad atención a que esta sentencia de control en C no requiere de un separador ; al final.

Y, ¿qué sucede si la entrada contiene algo como **while (10 - a) { a = a +1 ; } ?** Más adelante explicaremos cómo tratar estos casos.